

Modeling the System with UML

What you are building: a revised use-case diagram, a class diagram and ONE behaviour diagram (sequence OR activity) for the Smart Campus system from Week 03. An AI drafts each one; you review, correct and defend every element. **Time:** ~50 min in class + ~60 min at home. Full steps: week-04/README.md.

Your choice: the AI assistant and the model — nothing else. Freeze both for the whole lab and record the exact model name with its version ("ChatGPT" is not a model). There is no implementation language this week: the notation is PlantUML for everyone, because the checker reads it and everyone's diagrams must be comparable. Python runs the two checkers: Path A on your machine, Path B in a GitHub Codespace in the browser (Code → Codespaces → Create) — equal, and neither costs you anything.

Identical for everyone: the scenario and R1–R4 · the four prompts below, sent unchanged · the file names · the PlantUML conventions.

Scenario and rules (practice deck, slide 7)

Students view room availability, book a room, and cancel their own bookings. Administrators block or unblock rooms and review usage.

Rule	Text
R1	Future start, with duration greater than 0 and at most 2 hours.
R2	Active bookings for the same room cannot overlap.
R3	A blocked room cannot accept a new booking.
R4	A successful booking produces a confirmation.

Stories: your Week 03 stories as revised after review, or the reference set US-01...US-06 in models/approved-stories.md. Still undecided by the scenario — declare, do not guess: do touching bookings (12:00 end, 12:00 start) overlap? What happens to existing bookings when a room is blocked?

Part 0 — Setup (~4 min)

1. Branch week-04, copy the week-04/ folder, fill in models/approved-stories.md, commit.
2. Open ONE new chat. First message: the whole approved-stories.md plus this line:

This is the Smart Campus scenario, its rules R1-R4 and my approved user stories. I will ask you for several UML diagrams in PlantUML. Use only this scenario. Wait for my first request.

Parts 1–3 — draft, save the original, review, revise (~23 min in class)

For every diagram: save the AI's reply unedited to models/original/<name>.puml → render it → review → write your revision to models/<name>.puml → render to models/img/<name>.png → record findings → commit.

Task 1 — use case

Using the supplied scenario and approved stories, generate PlantUML for a use-case diagram. Include Student and Administrator outside a named system boundary. Model their goals, show justified associations, and list assumptions. Use include or extend only with a clear reason.

Task 2 — class

Create a UML domain class diagram in PlantUML for Smart Campus. Start with Student, Room, and Booking. Add attributes, appropriate operations, and association multiplicities. Add other classes only when requirements justify them. Explain each relationship and list assumptions. Avoid unjustified inheritance or composition.

Task 3A — sequence (or 3B — both is optional, earns nothing extra)

Generate PlantUML for Book room. Use Student, BookingService, and BookingRepository lifelines. Validate the supplied rules, then attempt the reservation. Show a successful confirmation and an unavailable-room alternative using alt. Label messages and replies. Explain new design components and all assumptions.

Task 3B — activity

Generate a UML activity diagram in PlantUML for Book room. Show the initial node, actions, guarded decisions, and final nodes. Check the time range, blocked-room status, and overlapping bookings. Show confirmation after success and rejection after failure. Use branches rather than parallel paths unless concurrency is required.

Review questions — this is what gets graded

- **Use case:** is each actor linked only to goals the scenario gives it? Does each use case trace to a story? Is it a goal, or a screen / component / system action?
- **Class:** domain concepts only (no Service, Repository, Screen). Read every association both ways: "one student makes 0..* bookings; each booking belongs to exactly 1 student". R2 cannot be a multiplicity — put it in a note.
- **Sequence:** validation before creation · nothing saved on a failure branch · every alt branch guarded · where are R1 and R3 visible?
- **Activity:** three decisions (R1, R3, R2), not one "all pass?" · labelled guards · no fork · create only after the last check.

PlantUML conventions

Rule	Example
' why: <reason> on the line directly above every include, extend, inheritance, composition, aggregation	' why: R4 - every successful booking confirms

Sequence guards WITHOUT square brackets (typed brackets become a hyperlink)	alt created ... else unavailable ... end
Activity guards WITH brackets inside parentheses	if (Room blocked? (R3)) then ([no])

Part 4 — critique and consistency (~10 min, at home)

Open a NEW chat (a critique in the same chat defends its own drafts). Paste approved-stories.md and your three revised .puml files, then send unchanged:

Compare my diagrams with the requirements. Identify missing rules, inconsistent names, and unjustified elements. Cite each issue and propose a specific correction.

Record ≥ 3 issues with accept / reject and a reason (§6). Fill in the consistency table (§7) and the change log (§8).

Part 5 — check, declare, submit (~10 min, at home)

```
cd week-04
python tests/check_models.py
git rev-parse --short HEAD
python tests/validate_submission.py
```

Verdict	Meaning
PASS	the check is satisfied
FAIL	the file is there and readable, but the check is not satisfied
ERROR	the file the check needs is missing or unreadable, so the check could not run

The checker checks shape, never quality. A fresh folder prints 1 PASS · 8 FAIL · 21 ERROR. Paste the full output in §9 and explain every FAIL you keep. Commit. Then fill in submission.yml last: the SUMMARY numbers exactly as printed, the commit hash you ran at, and every kept FAIL ID in checker.kept_fails. **Your numbers are re-run automatically at that commit — numbers the re-run does not reproduce cost the whole evidence criterion.** Totals differ by what you submitted — 30 with no behaviour diagram, 35 activity, 37 sequence, 43 both. A different total from your neighbour is not a lost check.

- 1. Branch week-04, ≥ 3 meaningful commits, including one made in class.
- 2. Pull Request week-04 → main in your own repository. Leave it open — do not merge. **Description: the five SETUP.md §4 headings, spelled exactly** (What I built · AI tools used · What the AI got wrong · Time spent · What I would do differently). Reported only in Week 03; checked automatically and counted from this week.
- 3. Teams assignment: + Add work → Link → paste the PR URL. Not the repo URL.

Deadline: before the start of the next practice session. Late per course policy; Teams closes at the end of the following week.

Grading — 1 point, 4 × 0.25

Criterion	0.25 when ...
Diagrams	Revised use-case, class and one behaviour diagram render from source; originals kept unedited; an image for each.
Review	§3–§5 findings name the element, the problem and the rule or story; associations read both ways; R2 note; assumptions declared.
Critique & consistency	≥ 3 critique issues with a reasoned verdict; R1–R4 and every use case traced; ≥ 3 real changes logged, one per diagram.
Evidence & honesty	Full checker output with every kept FAIL explained; AI_USAGE.md complete; submission.yml validates and the re-run at your commit reproduces its numbers; PR open with the five headings, ≥ 3 commits.

A FAIL you report and explain costs you nothing. One you hide costs the whole criterion. Worthless: "the AI got the multiplicities wrong". Worth everything: "the AI put 1..* on the Booking end of Room–Booking, which says every room must already have a booking".

Not accepted: declared numbers the re-run does not reproduce · images without .puml source · edited originals · edited or shortened checker output · placeholder <...> rows · findings that cite no element and no rule · a merged PR, a repo link, or an emailed submission.